Setting Up Your F1 Database in Docker

This guide will help you set up a complete environment to collect F1 data and view it in a database using Docker.

Step 1: Set Up Project Structure

First, create a new directory for your project and set up the following files:

```
f1-database-project/
├── docker-compose.yml
├── Dockerfile
├── init.sql
├── f1_data_collector.js
└── package.json
```

Step 2: Create Docker Configuration Files

docker-compose.yml

yaml

```yaml
version: '3'

services:
  # PostgreSQL database
  postgres:
    image: postgres:14
    container_name: f1-database
    environment:
      POSTGRES_USER: f1user
      POSTGRES_PASSWORD: f1password
      POSTGRES_DB: f1data
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql/data
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql
    networks:
      - f1network

  # pgAdmin for database management
  pgadmin:
    image: dpage/pgadmin4
    container_name: f1-pgadmin
    environment:
      PGADMIN_DEFAULT_EMAIL: user@example.com
      PGADMIN_DEFAULT_PASSWORD: pgadmin
    ports:
      - "8080:80"
    depends_on:
      - postgres
    networks:
      - f1network

  # Node.js application to collect F1 data
  app:
    build: .
    container_name: f1-data-collector
    volumes:
      - .:/app
    depends_on:
      - postgres
    networks:
      - f1network

networks:
  f1network:
    driver: bridge

volumes:
  postgres-data:
```

Dockerfile

dockerfile

```
FROM node:18

WORKDIR /app

COPY package.json ./
RUN npm install

COPY . .

CMD ["node", "f1_data_collector.js"]
```

package.json

json

```
{
  "name": "f1-database-project",
  "version": "1.0.0",
  "description": "F1 Database Collector",
  "main": "f1_data_collector.js",
  "scripts": {
    "start": "node f1_data_collector.js"
  },
  "dependencies": {
    "pg": "^8.11.0",
    "node-fetch": "^3.3.1"
  },
  "type": "module"
}
```

init.sql

```sql
-- Create tables for F1 database

-- Constructors table
CREATE TABLE constructors (
    constructor_id INTEGER PRIMARY KEY,
    constructor_name TEXT NOT NULL
);

-- Drivers table
CREATE TABLE drivers (
    driver_id INTEGER PRIMARY KEY,
    driver_name TEXT NOT NULL,
    team_name TEXT,
    country_code TEXT
);

-- Tracks table
CREATE TABLE tracks (
    track_id INTEGER PRIMARY KEY,
    track_name TEXT NOT NULL,
    circuit_key TEXT NOT NULL,
    country_name TEXT,
    location TEXT
);

-- Race results table
CREATE TABLE race_results (
    race_id INTEGER PRIMARY KEY,
    meeting_key TEXT NOT NULL,
    circuit_key TEXT NOT NULL,
    date TEXT NOT NULL,
    year INTEGER NOT NULL,
    quali_positions TEXT NOT NULL,
    race_positions TEXT NOT NULL
);
```

Step 3: Create F1 Data Collector Script

f1_data_collector.js

javascript

```
// Using ES modules
import fetch from 'node-fetch';
import pg from 'pg';

const { Pool } = pg;

// Configure PostgreSQL connection
const pool = new Pool({
  user: 'f1user',
  host: 'postgres',
  database: 'f1data',
  password: 'f1password',
  port: 5432,
});

// Function to fetch data from OpenF1 API
async function fetchOpenF1Data(endpoint, params = {}) {
  const baseUrl = "https://api.openf1.org/v1/";
  const url = new URL(baseUrl + endpoint);

  // Add query parameters
  Object.keys(params).forEach(key => {
    if (params[key] !== undefined) {
      url.searchParams.append(key, params[key]);
    }
  });

  try {
    const response = await fetch(url);
    if (!response.ok) {
      throw new Error(`API request failed with status: ${response.status}`);
    }
    return await response.json();
  } catch (error) {
    console.error(`Error fetching ${endpoint}:`, error);
    return [];
  }
}

// Function to get constructor data
async function getConstructors(year) {
  const drivers = await fetchOpenF1Data("drivers", { year });

  // Extract unique constructor (team) data
  const constructorsMap = new Map();
  let constructorId = 1;

  drivers.forEach(driver => {
    if (driver.team_name && !constructorsMap.has(driver.team_name)) {
      constructorsMap.set(driver.team_name, {
        constructor_id: constructorId++,
        constructor_name: driver.team_name
      });
    }
  });

  return Array.from(constructorsMap.values());
}

// Function to get driver data
async function getDrivers(year) {
  const drivers = await fetchOpenF1Data("drivers", { year });

  // Process driver data
```

```
  return drivers.map(driver => ({
    driver_id: driver.driver_number,
    driver_name: driver.full_name,
    team_name: driver.team_name,
    country_code: driver.country_code
  }));
}

// Function to get track data
async function getTracks(year) {
  const meetings = await fetchOpenF1Data("meetings", { year });

  // Process track data
  let trackId = 1;
  return meetings.map(meeting => ({
    track_id: trackId++,
    track_name: meeting.circuit_short_name,
    circuit_key: meeting.circuit_key,
    country_name: meeting.country_name,
    location: meeting.location
  }));
}

// Function to get qualifying positions for a specific meeting
async function getQualifyingPositions(meetingKey) {
  // Find qualifying session key
  const sessions = await fetchOpenF1Data("sessions", { meeting_key: meetingKey });
  const qualifyingSession = sessions.find(s => s.session_name && s.session_name.includes("Quali

  if (!qualifyingSession) return [];

  // Get final qualifying positions
  const positions = await fetchOpenF1Data("position", {
    session_key: qualifyingSession.session_key
  });

  // Group by driver and get their final position
  const driverPositions = new Map();

  positions.forEach(pos => {
    driverPositions.set(pos.driver_number, pos.position);
  });

  // Convert to array of {constructor_id, driver_id} objects
  const result = [];

  for (let [driverNumber, position] of driverPositions.entries()) {
    // We'll populate the constructor_id later when combining data
    result[position-1] = { driver_id: driverNumber };
  }

  return result;
}

// Function to get race positions for a specific meeting
async function getRacePositions(meetingKey) {
  // Find race session key
  const sessions = await fetchOpenF1Data("sessions", { meeting_key: meetingKey });
  const raceSession = sessions.find(s => s.session_name && s.session_name.includes("Race"));

  if (!raceSession) return [];

  // Get final race positions
  const positions = await fetchOpenF1Data("position", {
    session_key: raceSession.session_key
```

```
  });

  // Group by driver and get their final position
  const driverPositions = new Map();

  positions.forEach(pos => {
    driverPositions.set(pos.driver_number, pos.position);
  });

  // Convert to array of {constructor_id, driver_id} objects
  const result = [];

  for (let [driverNumber, position] of driverPositions.entries()) {
    // We'll populate the constructor_id later when combining data
    result[position-1] = { driver_id: driverNumber };
  }

  return result;
}

// Function to insert data into database
async function insertData(table, data) {
  if (!data || data.length === 0) {
    console.log(`No data to insert into ${table}`);
    return;
  }

  // Get column names from the first object
  const columns = Object.keys(data[0]);

  for (const item of data) {
    const values = columns.map(col => item[col]);
    const placeholders = columns.map((_, i) => `$${i+1}`).join(', ');

    const query = {
      text: `INSERT INTO ${table} (${columns.join(', ')}) VALUES (${placeholders}) ON CONFLICT
      values: values,
    };

    try {
      await pool.query(query);
    } catch (error) {
      console.error(`Error inserting into ${table}:`, error);
      console.error('Query:', query);
    }
  }

  console.log(`Inserted ${data.length} rows into ${table}`);
}

// Main function to build and populate database
async function buildF1Database(year) {
  console.log(`Building F1 database for ${year}...`);

  // Get base data
  const constructors = await getConstructors(year);
  const drivers = await getDrivers(year);
  const tracks = await getTracks(year);

  // Insert base data
  await insertData('constructors', constructors);
  await insertData('drivers', drivers);
  await insertData('tracks', tracks);

  // Build constructor lookup map
```

```
  const constructorLookup = new Map();
  constructors.forEach(c => {
    constructorLookup.set(c.constructor_name, c.constructor_id);
  });

  // Build driver team lookup map
  const driverTeamLookup = new Map();
  drivers.forEach(d => {
    driverTeamLookup.set(d.driver_id, d.team_name);
  });

  // Get all meetings for the year
  const meetings = await fetchOpenF1Data("meetings", { year });
  console.log(`Found ${meetings.length} meetings for ${year}`);

  // Process each meeting to get race data
  const raceResults = [];
  let raceId = 1;

  for (const meeting of meetings) {
    console.log(`Processing meeting: ${meeting.meeting_name || meeting.meeting_key}`);

    // Get qualifying and race positions
    const qualiPositions = await getQualifyingPositions(meeting.meeting_key);
    const racePositions = await getRacePositions(meeting.meeting_key);

    // Add constructor_id to each position
    qualiPositions.forEach(pos => {
      const teamName = driverTeamLookup.get(pos.driver_id);
      pos.constructor_id = constructorLookup.get(teamName);
    });

    racePositions.forEach(pos => {
      const teamName = driverTeamLookup.get(pos.driver_id);
      pos.constructor_id = constructorLookup.get(teamName);
    });

    // Add to race results
    raceResults.push({
      race_id: raceId++,
      meeting_key: meeting.meeting_key,
      circuit_key: meeting.circuit_key || '',
      date: meeting.date_start || new Date().toISOString(),
      year: meeting.year || year,
      quali_positions: JSON.stringify(qualiPositions),
      race_positions: JSON.stringify(racePositions)
    });
  }

  // Insert race results
  await insertData('race_results', raceResults);

  console.log('Database build complete!');
}

// Wait for PostgreSQL to be ready before starting
async function waitForPostgres() {
  let retries = 10;
  while (retries > 0) {
    try {
      const client = await pool.connect();
      client.release();
      console.log('Connected to PostgreSQL!');
      return true;
    } catch (err) {
```

```
      console.log('Waiting for PostgreSQL to start...', retries);
      retries--;
      await new Promise(resolve => setTimeout(resolve, 3000));
    }
  }
  return false;
}

// Main execution
async function main() {
  try {
    const isPostgresReady = await waitForPostgres();

    if (!isPostgresReady) {
      console.error('Failed to connect to PostgreSQL after multiple attempts. Exiting.');
      process.exit(1);
    }

    // Build database for 2023 season
    await buildF1Database(2023);

    console.log('F1 data collection complete!');
  } catch (error) {
    console.error('Fatal error:', error);
  } finally {
    await pool.end();
  }
}

main();
```

Step 4: Run Your Docker Setup

1. Open a terminal and navigate to your project directory:

```bash
cd f1-database-project
```

2. Build and start the Docker containers:

```bash
docker-compose up -d
```

3. Check that all containers are running:

```bash
docker-compose ps
```

You should see three containers: f1-database (PostgreSQL), f1-pgadmin (pgAdmin), and f1-data-collector (Node.js app).

4. Check the logs to see the data collection progress:

```bash
docker logs -f f1-data-collector
```

Step 5: Access and View Your F1 Data

1. Open your web browser and go to: `http://localhost:8080`
2. Log in to pgAdmin using:

- Email: `user@example.com`
- Password: `pgadmin`

3. Add a new server connection in pgAdmin:
   - Name: `F1 Database`
   - Connection tab:
     - Host: `postgres`
     - Port: `5432`
     - Maintenance database: `f1data`
     - Username: `f1user`
     - Password: `f1password`
4. Click "Save" to connect to your database
5. Browse your data:
   - Expand: Servers → F1 Database → Databases → f1data → Schemas → public → Tables
   - Right-click on any table (e.g., "constructors") and select "View/Edit Data" → "All Rows"

Step 6: Example Queries

Here are some example queries you can run in pgAdmin to view your data:

View all constructors:

```sql
SELECT * FROM constructors;
```

View all drivers with their teams:

```sql
SELECT d.driver_id, d.driver_name, d.team_name, c.constructor_id
FROM drivers d
LEFT JOIN constructors c ON d.team_name = c.constructor_name;
```

View qualifying positions for a specific race:

```sql
SELECT
    r.race_id,
    t.track_name,
    r.date,
    jsonb_array_elements(r.quali_positions::jsonb) as quali_position
FROM race_results r
JOIN tracks t ON r.circuit_key = t.circuit_key
WHERE t.track_name = 'Bahrain';
```

Compare qualifying and race positions:

```sql
SELECT
    t.track_name,
    d.driver_name,
    (jsonb_array_elements(r.quali_positions::jsonb) ->> 'driver_id')::int as driver_id,
    jsonb_array_position(r.quali_positions::jsonb, jsonb_array_elements(r.quali_positions::jsonb)
    jsonb_array_position(r.race_positions::jsonb, jsonb_array_elements(r.race_positions::jsonb))
FROM race_results r
JOIN tracks t ON r.circuit_key = t.circuit_key
JOIN drivers d ON (jsonb_array_elements(r.quali_positions::jsonb) ->> 'driver_id')::int = d.dri
WHERE t.track_name = 'Monaco';
```

Troubleshooting

Issue: Container fails to start

- Check logs: `docker logs f1-database`
- Ensure ports are available: Make sure nothing else is using ports 5432 or 8080

Issue: Data not appearing

- Check app logs: `docker logs f1-data-collector`
- Try running a specific year: Edit f1_data_collector.js to try a different year (e.g., 2022)

Issue: pgAdmin can't connect

- Check network: Ensure all containers are on the same network
- Verify credentials: Double-check username and password

Issue: Need to reset everything

- Stop and remove all containers, volumes, and networks:

```bash
docker-compose down -v
docker-compose up -d
```